



Java程序设计

Ch03-3 抽象类与接口

余 烨

本章学习提示

- 本章学习内容包括
  - 什么是抽象类接口，它们的作用与异同
  - 什么是简单工厂模式？如何使用
  - == 和 equals()
  - 函数调用时参数的传值与可变参数

目　录

1 抽象类 与 接口

2 “设计模式”初体验 – 简单工厂模式

3 ==和java.lang.Object.equals()

4 函数的调用

抽象方法

- 只有方法说明，而没有方法体的方法称为抽象方法
- 抽象方法的定义

  abstract <类型> 方法名 ([参数表]);
- 例：

```
class Robot
{
    String name;     //成员变量
    abstract void Walk();    //抽象方法
}
```

```
void Walk() {}
//空方法体的方法
```

抽象类

- 抽象类是用修饰符*abstract*说明的类
- 定义抽象类的一般形式

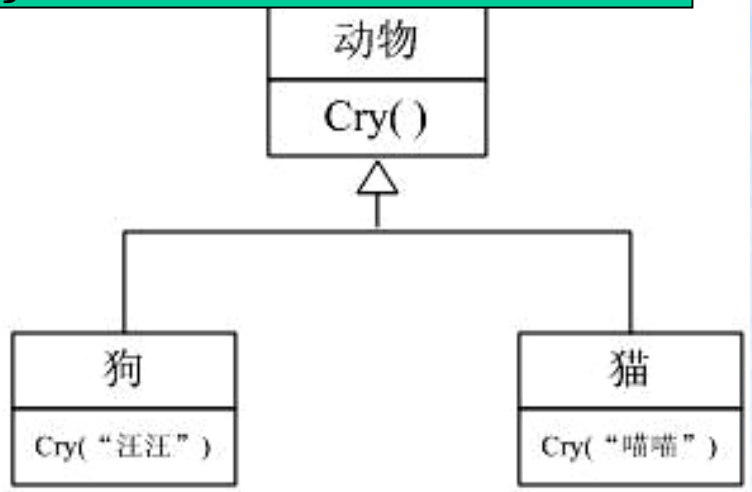
```
abstract  class  类名称
{
        成员变量;
        方法( ) { /* do something*/ };    //定义一般方法
        abstract    方法( );  //定义抽象方法
}
```

- 抽象类通常都包括一个或多个抽象方法。

抽象类 例1

```
class 动物 {
    void cry( ) {
        System.out.println("不知道怎么叫！"); }
}
class 狗 extends 动物 {
    void cry() {
        System.out.println("汪汪....."); }
}
class 猫 extends 动物 {
    void cry() {
        System.out.println("喵喵....."); }
}
public class TestCry {
    public static void main(String args[]) {
        动物 dongwu;
        dongwu=new 狗();
        dongwu.cry();
        dongwu=new 猫();
        dongwu.cry();
    } }
```

```
abstract class 动物{
    abstract void cry();
}
```



动物 dongwu = new 动物();



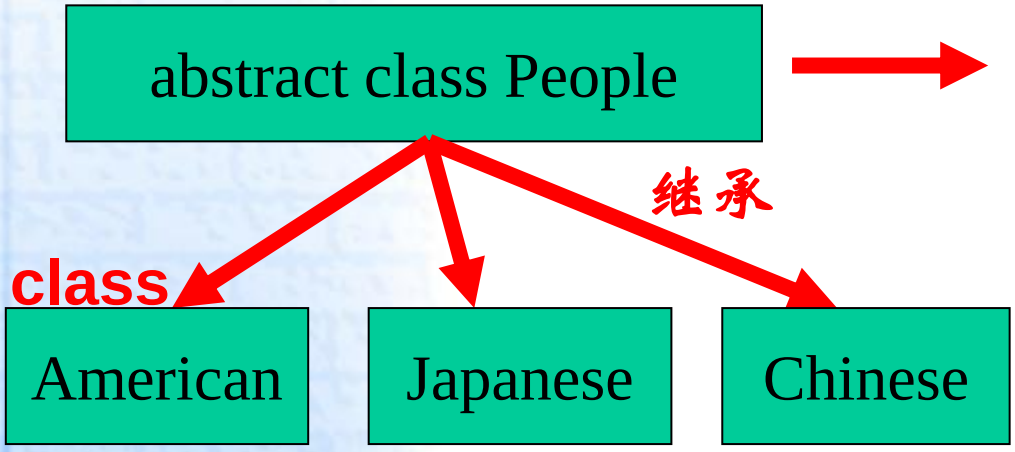
TestAbstractClass.java

抽象类

- 特点：抽象类没有具体的对象，不能进行实例化



概念类



People someguy;



People someguy
= new People();



Chinese ChGuy = new Chinese();

American AmGuy = new American();

Japanese JpGuy = new Japanese();

为什么要有抽象类

- 作用：
  - 抽象类不能用来创建对象，只能被当作父类被子类继承
  - 用来抽象，作为很多具有共同子类的父类。
  - 抽象类的应用，会有多态性的体现。

> “当我看到类是抽象的，我就很关心它的抽象方法。我知道它的子类一定会重写它，而且，我会去找到抽象类的引用，它一定会有多态性的体现。”

- 从多个具有相同特征的类中抽象出一个抽象类，作为父类，以这个类作为其子类的模版，从而避免了设计的随意性。子类在抽象类的基础上进行扩展、改造，但子类总体上会大致保留抽象类的行为方法。

示 例1

- 雇员示例
  - 需求：公司中程序员有姓名、工号、薪水、工作内容；项目经理除了有姓名、工号、薪水、工作内容，还有奖金。对给出需求的数据建模。
  - 分析（提炼类）：
    - 程序员：姓名、工号、薪水；工作
    - 项目经理：姓名、工号、薪水、奖金；工作
  - 程序员和项目经理不存在直接的继承关系，但是程序员和项目经理却有共性的内容，可以进行抽取，因为他们都是公司的雇员。

TestEmployer

```
abstract class Employee
{
        private String name;
        private String id;
        private double pay;
        Employee() {   }
        Employee(String name, String id, double pay)
        {
                this.name = name;
                this.id = id;
                this.pay = pay;
        }
        //不清楚具体怎么工作，所以用抽象函数来定义
        public  abstract void work();
}
```

```
class Programmer extends Employee {
        Programmer(String name, String id, double pay) {
                super(name,id,pay);
        }
        public void work()    {
                System.out.println("code...");
        }
}
```

```
class Manager extends Employee {
        private int bonus;
        Manager(String name, String id, double pay, int bonus) {
                super(name, id, pay);
                this.bonus = bonus;
        }
        public void work()    {
                System.out.println("manage...");
        }
}
```

```
public class TestEmployer
{
        public static void main(String[] args)
        {
                Employee emp[] = new Employee[2];
                emp[0] = new Manager("Jack","1",8000.5, 2000);
                emp[0].work();
                emp[1] = new Programmer("John", "2", 6000.0);
                emp[1].work();
        }
}
```

其他说明

- **说明：**
  - 抽象类 里 可以有 非抽象的方法；
  - 可以没有 抽象方法（但很少见，目的就是不让该类创建对象）；即使没有抽象方法的抽象类也不允许实例化。
  - 抽象类中可以有构造函数（目的是为了给子类进行初始化）；
  - 如果一个类里有抽象方法，则该类必须被声明为抽象类。（一般类中不能有抽象方法）
  - 抽象方法一定不能有方法体（哪怕是空方法体）。
  - 当继承一个抽象类时：可以不重新定义部分或全部的抽象方法，这时必须把子类也显式定义为抽象类；可以重新定义所有的抽象方法，这样子类就不必被定义为抽象类。

注 意

- 当abstract修饰类时，表明这个类只能被继承，当abstract修饰方法时，表明这个方法必须由子类提供实现（即重写）。而final修饰的类不能被继承，final修饰的方法不能被重写，因此，abstract不能和final同时使用。
- 当使用static来修饰一个方法时，表明这个方法是类方法，即通过类来调用，如果该方法被定义为抽象方法，则将导致通过该类来调用该方法时出现错误（调用一个没有方法体的方法是无意义的），因此，abstract不能和static同时使用。
- abstract关键字修饰的方法必须被其子类重写才有意义，否则这个方法将永远不会有方法体，因此abstract方法不能定义为private访问权限，因为private在子类中不可见，无法被重写，即abstract不能和private同时使用。
- abstract不能用于修饰属性，不能用于修饰局部变量，即没有抽象变量、没有抽象属性等说法；abstract也不能用于修饰构造方法，没有抽象构造方法。抽象类里定义的是普通的构造方法。

```
public abstract class Animal  {
        public abstract void run(); //抽象方法
        public void breath()  {
                System.out.println("呼吸");
                run();     请问此处的调用是否正确？输出什么？
                }
        public static void main(String[] args)  {
                Cat cat1 = new Cat();
                cat1.breath();
        }
}
class Cat extends Animal   {
        public void run()    {
                System.out.println("猫跑");
        }
}
```

Animal.java

接口 – 从USB接口说起…





要设计几个类？

每个类都要实现
USB接口这一规范





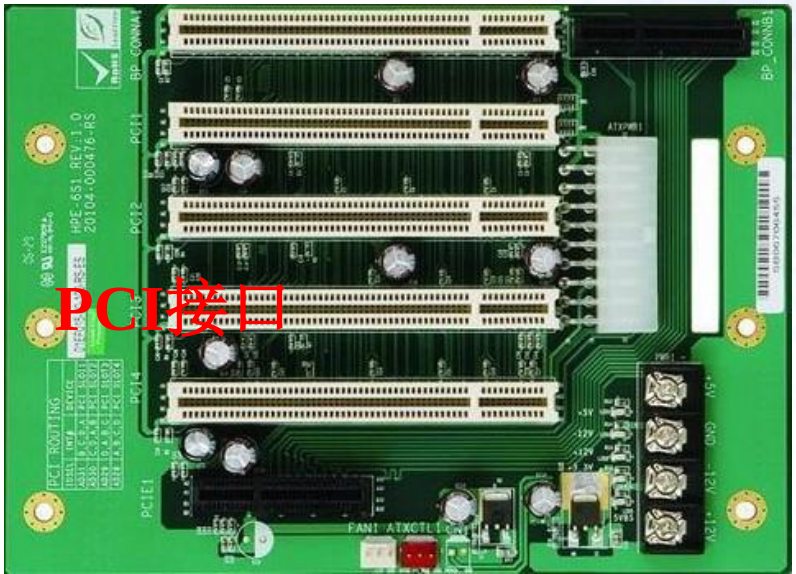


接 口

- 类是一种具体实现体，包含属性和行为；而接口定义了一种规范，接口定义某一批类所需要遵守的规范，接口不关心这些类的内部状态数据，也不关心这些类里方法的实现细节。它只规定这批类里必须提供某些方法，提供这些方法的类就可满足实际需要。
- 接口体现的是规范和实现分离的设计哲学

↓

让软件系统的各组件之间面向接口耦合，是一种松耦合的设计



接口定义

- Java中接口定义的一般形式如下：

  [访问控制符] *interface* 接口名 {

  成员变量 全局变量，默认修饰符：public static final

  抽象方法

  } 默认修饰符：public abstract

- 使用**implements**关键字实现一个接口
- 在实现一个接口时，就要求实现接口中所有的方法

```
interface Demo
{
        int ID = 4;
        void show1();
        void show2();
}
```

```
class InterfaceDemo implements Demo
{
                public void show1()  {      }
                public void show2()  {      }
}
```

接口定义的说明

- 接口不能被实例化。只能由实现了接口的子类并覆盖了接口中所有的抽象方法后，该子类才可以实例化

  interface USB{……}

  USB usb = new USB();



- 和类的访问级别类似，接口的访问控制符是 public 或者default。
  - 如果接口本身定义成public，所有抽象方法和成员变量都是public的，不论是否使用了public访问修饰符
- 接口中的所有方法都是抽象方法(没有方法体)（类似C++的“纯虚类”），默认修饰符为public abstract。在实现接口中的方法时，一定要将实现后的方法声明为public。当某个类实现一个接口时，就要求实现接口中所有的方法。

接口的说明

- 接口中可以有成员变量（可以省略访问控制修饰符，但如果指定，只能使用public）
  - 接口中的变量，默认就是public static final，必须以常量值初始化

```
//系统自动为接口里定义属性增加public static final修饰符
  int MAX_SIZE = 50;
  public static final int MAX_SIZE = 50;
```

  - 实际开发中，常在接口中定义常用的变量，作为全局变量使用，访问形式：接口名.变量名
  - 从某个角度看，接口可以被当成一种特殊的类，因此，一个java源文件里最多只能有一个public接口，且public接口也必须定义在与接口同名的文件中

TestExample

```
package lee;
public interface Output {
        //接口里定义的变量是常量
        int MAX_CACHE_LINE = 50;
        //接口里定义的只能是public的抽象实例方法
        void out();
        void getData(String msg);  }
```




```
package yeeku;
import java.lee.Output;
public class TestOutputProperty
{
        public static void main(String[] args) {
            //访问Output接口的属性
            System.out.println(lee.Output.MAX_CACHE_LINE);
            lee.Output.MAX_CACHE_LINE = 20;
           //上面的语句将引起编译异常
} }
```

接口的说明

- 形式最简单的接口可以不包含任何常量和方法定义
- 接口中只可能有常量和抽象方法
- 一个接口不能继承其它的类，但是可以继承别的接口。接口完全支持多继承。
- 一个类可以实现多个接口
- 一个类在继承另一个类的同时，还可以实现多个接口（接口的出现避免了单继承的局限性）

```
interface interfaceA
{  int PROP_A = 5;
    void testA();
}
```

```
interface interfaceB
{  int PROP_B = 6;
    void testB();
}
```

```
interface interfaceC extends interfaceA, interfaceB
{  int PROP_C = 7;
    void testC();
}
```

```
class  classA implements interfaceA, interfaceB
{
      public void testA() {System.out.println("testA is run");}
      public void testB() {System.out.println("testB is run");}
}
```

```
Public class TestInterfaceExtends
{
    public static void main(String[] args)
    {
        System.out.println(interfaceC.PROP_A);
        System.out.println(interfaceC.PROP_B);
        System.out.println(interfaceC.PROP_C);
        classA  object1 = new classA();
        System.out.println(object1.PROP_A);
        System.out.println(object1.PROP_B);
        object1.testA();
        object1.testB();
    }   }
```

接口引用与实现对象的关系

- 同子类对象可以赋给父类引用一样，**接口引用变量可以指向实现对象**。
  - 可以定义一个接口类型的引用变量，**并将任何实现了该接口的类的对象赋给它**。当通过接口引用变量调用方法时，将根据**动态绑定**的原则来决定方法的正确调用。
  - 需要注意的是，虽然实现对象可以赋给接口引用，但该引用**只能访问被它的接口定义声明的内容**，而隐藏实现对象中的特殊内容。

接口、实现与继承例



USB接口　　Bluetooth接口

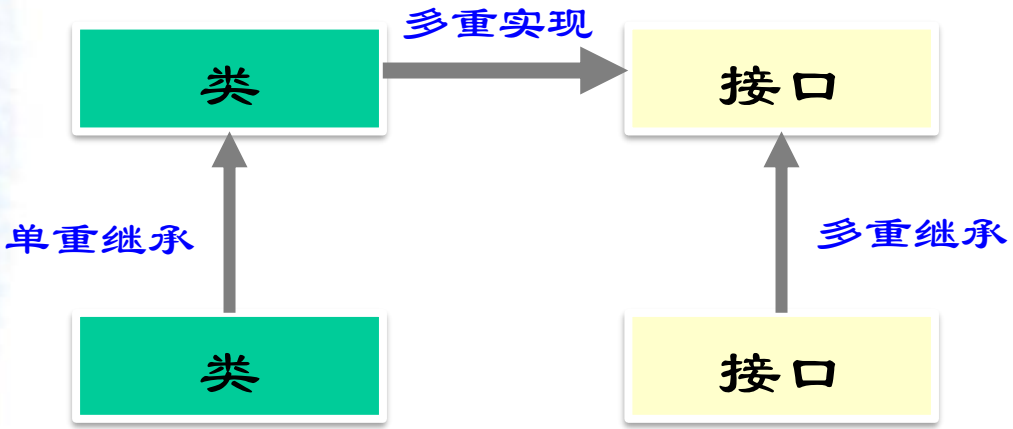
凡是有USB接口、蓝牙的设备都能使用

对上述现象进行描述……

TestUSB.java

abstract class和interface

- 从语法的角度
  - 一个类实现一个接口，使用关键字implements.
  - 一个类可以实现(implements)多个接口、一个接口可以继承(extends)多个接口；



abstract class和interface

- 从设计层面（Effective java》）

1. 接口比抽象类好，一般情况下，如果能用接口就不用抽象类
2. 如果一个子类继承了抽象类就决定了这个类的主要特征，接口表示实现类**尊崇接口的“协议”，并不是接口的特征**
3. 接口可以将不相关的类通过相同的某种行为联系到一起，抽象类则不能
4. 类只能使用单继承，而接口可以继承多个接口，类可以实现多个接口，它使设计更加灵活

抽象类 VS 接口

	区别点	抽象类	接口
1	定义	abstract修饰的类	抽象方法和全局常量的集合
2	组成	构造方法、抽象方法、普通方法、常量、变量	抽象方法、常量
3	使用	子类继承抽象类（extends）	子类实现接口（implements）
4	关系	抽象类可以实现多个接口	接口不能继承抽象类，但允许实现多个接口
5	设计模式	模板设计	工厂设计、代理设计
6	对象	都通过对象的多态性产生实例化对象	
7	局限	抽象类有单继承的局限	接口没有此局限
8	实际使用	作为一个模板	是作为一个标准或是表示一种能力
9	选择	**若抽象类和接口都可以使用，优先使用接口，避免单继承的问题**	

主观题 20分

设置

犬类中，导盲犬、搜爆犬；

猪类中，也有导盲猪、搜爆猪

……

请用接口、抽象类实现这一描述。

正常使用主观题需2.0以上版本雨课堂

作答

目　录

1 抽象类 与 接口

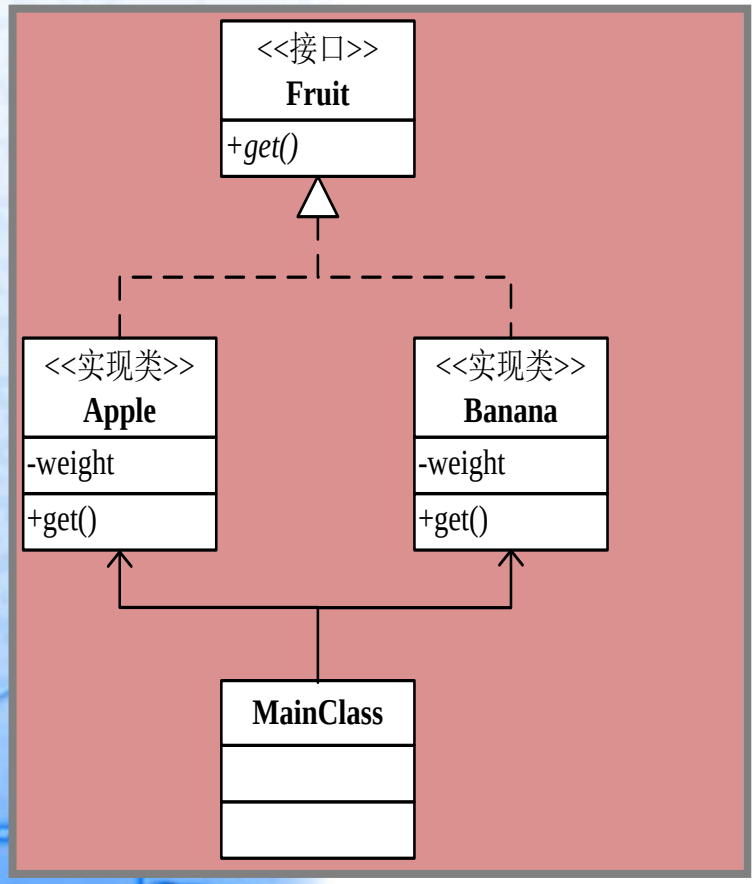
2 “设计模式”初体验 – 简单工厂模式

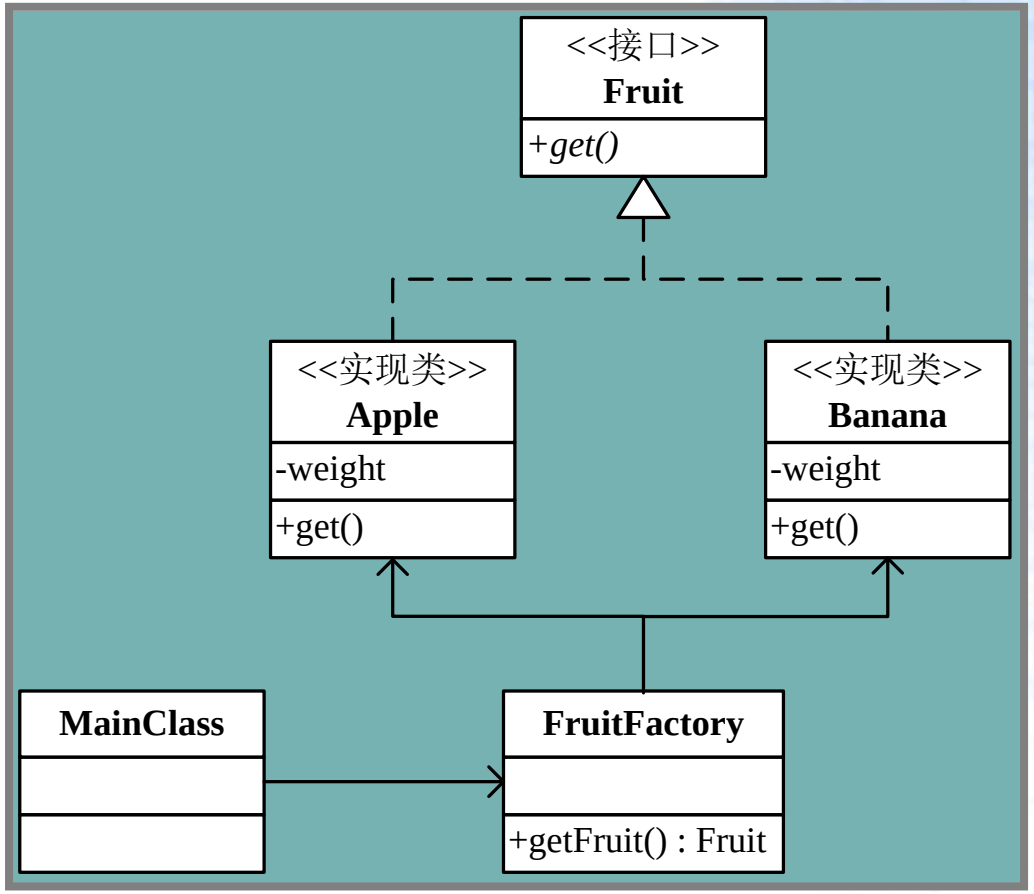
3 ==和java.lang.Object.equals()

4 函数的调用

工厂模式

- 简单工厂模式属于类的创建型模式，又叫做静态工厂方法模式。通过专门定义一个类来负责创建其他类的实例，被创建的实例通常都具有共同的父类。





示例 – 水果类

```
interface Fruit {
    //采集
    public void get();
}
class Apple implements Fruit{
    //采集
    public void get(){
        System.out.println("采集苹果");
    }
}
class Banana implements Fruit{
    //采集
    public void get(){
        System.out.println("采集香蕉");
    }
}
```

SimpleFactory_水果之例

示例——简单工厂类

```
class FruitFactory {
    //get方法，获得所有产品对象
    public static Fruit getFruit( String type ) throws Exception {
        Fruit fruit = null;
        if ( type.equalsIgnoreCase( "apple" ) ) {
          fruit = new Apple();
          return fruit;
        } else if ( type.equalsIgnoreCase("banana") ) {
          fruit = new Banana();
          return fruit;
        } else {
          System.out.println("找不到相应的实例化类");
          return null;
        }
    }
}
```

示例——主类

```
public class MainClass {
    public static void main(String[] args)  throws Exception {
        //简单工厂方法创建水果对象
        Fruit apple = FruitFactory.getFruit("Apple");
        Fruit banana = FruitFactory.getFruit("Banana");
        //调用水果对象方法
        apple.get();
        banana.get();
    }
}
```

模式中包含的角色及其职责

- **抽象（Product）角色**

  简单工厂模式所创建的所有对象的父类，它负责描述所有实例所共有的公共接口。

- **具体产品（Concrete Product）角色**

  简单工厂模式所创建的具体实例对象

- **工厂（Creator）角色**

  简单工厂模式的核心，它负责实现创建所有实例的内部逻辑。工厂类可以被外界直接调用，创建所需的产品对象。

简单工厂模式的优缺点

优点

- 在这个模式中，工厂类是整个模式的关键所在。
- 它包含必要的判断逻辑，能够根据外界给定的信息，决定究竟应该创建哪个具体类的对象。用户在使用时可以直接根据工厂类去创建所需的实例，无需了解对象是如何创建以及如何组织的。
- 有利于整个软件体系结构的优化。

缺点

- 简单工厂模式的缺点也正体现在其工厂类。
- 工厂类集中了所有实例的创建逻辑，“高内聚”方面不佳。
- 另外，当系统中的具体产品类不断增多时，可能会出现要求工厂类也要做相应的修改，扩展性并不很好。

目　录

1 抽象类 与 接口

2 “设计模式”初体验 – 简单工厂模式

3 ==和java.lang.Object.equals()

4 函数的调用

“比 较”

- 操作符 ==
  - 当操作符==两边都是基本数值类型变量时，比较数值是否相等
  - 当操作符==两边都是引用类型变量时，这两个引用变量必须都引用同一个对象，结果才为true。
- Object类的equals()方法
  - 比较规则为：当参数obj引用的对象与当前对象为同一个对象，就返回true，否则返回false：

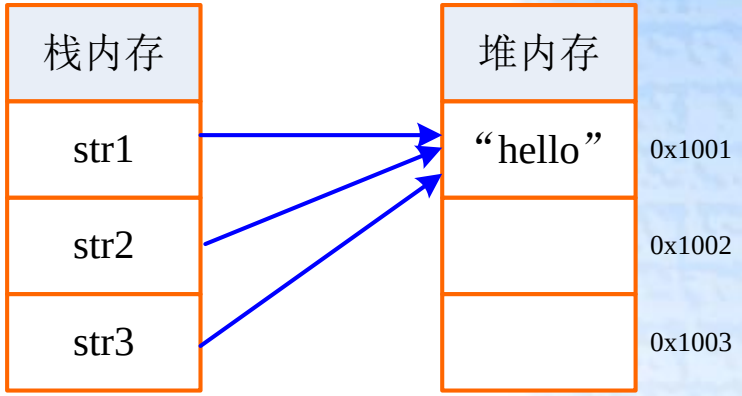
```
public boolean equals(Object obj) {
    if(this==obj)return true;
    else return false;
}
```

“==” 比较两个变量的内存空间是否一样；

“equals”比较两个对象变量所代表的对象的内容是否相等

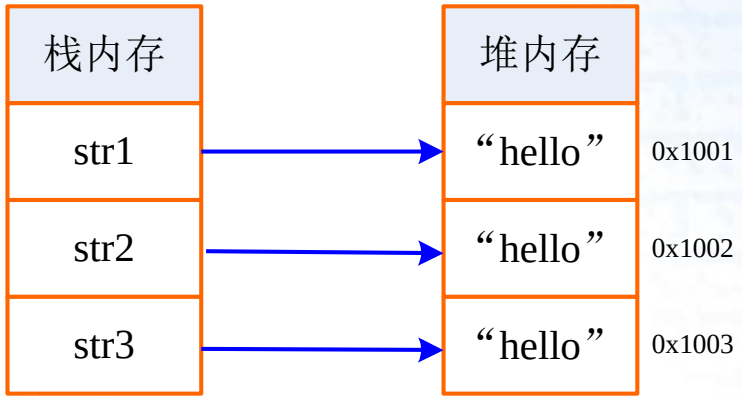
String类的比较

- 创建字符串的方式一：
  - String str1 = "hello" ;
    // 直接赋值
  - String str2 = “hello” ;
    // 直接赋值
  - String str3 = "hello" ;
    // 直接赋值



== ？ ✓　　equals ？ ✓

- 创建字符串的方式二：
  - String str1 = new String("hello")
    // 通过new赋值
  - String str2 = new String("hello")
    // 通过new赋值
  - String str3 = "hello" ;
    // 直接赋值



== ？ ✗　　equals ？ ✓

equals()方法

- 在JDK中有一些类覆盖了Object类的equals()方法，这些类包括：java.io.File、java.util.Date、java.lang.String、包装类（如java.lang.Integer和java.lang.Double类）。

```
Integer int1=new Integer(1);
Integer int2=new Integer(1);

String str1=new String("Hello");
String str2=new String("Hello");

System.out.println(int1==int2);
System.out.println(int1.equals(int2));

System.out.println(str1==str2);
System.out.println(str1.equals(str2));
```






训练题

- 定义一个Cat的类：
  - Cat有自己的名字和颜色；
  - 有构造函数，用于成员变量的赋值；
  - 覆盖Object类中的equals()方法，用于比较猫对象的内容是否一致
- 实例化两个猫的对象，比较其值是否相等。

TestEqual.java

目　录

1 抽象类 与 接口

2 “设计模式”初体验 – 简单工厂模式

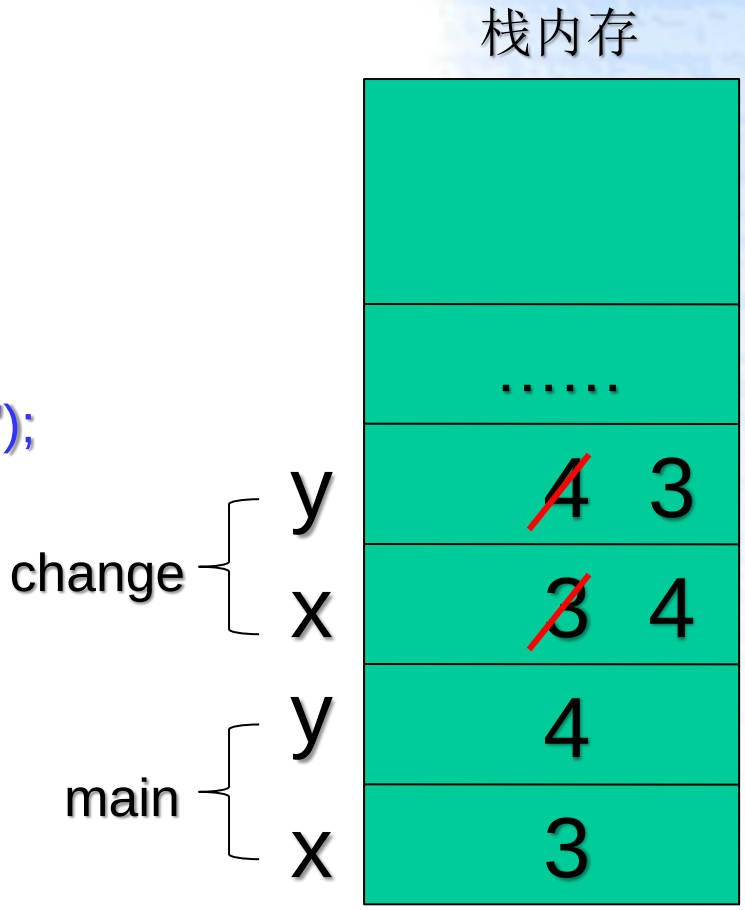
3 ==和java.lang.Object.equals()

4 函数的调用

参数传值

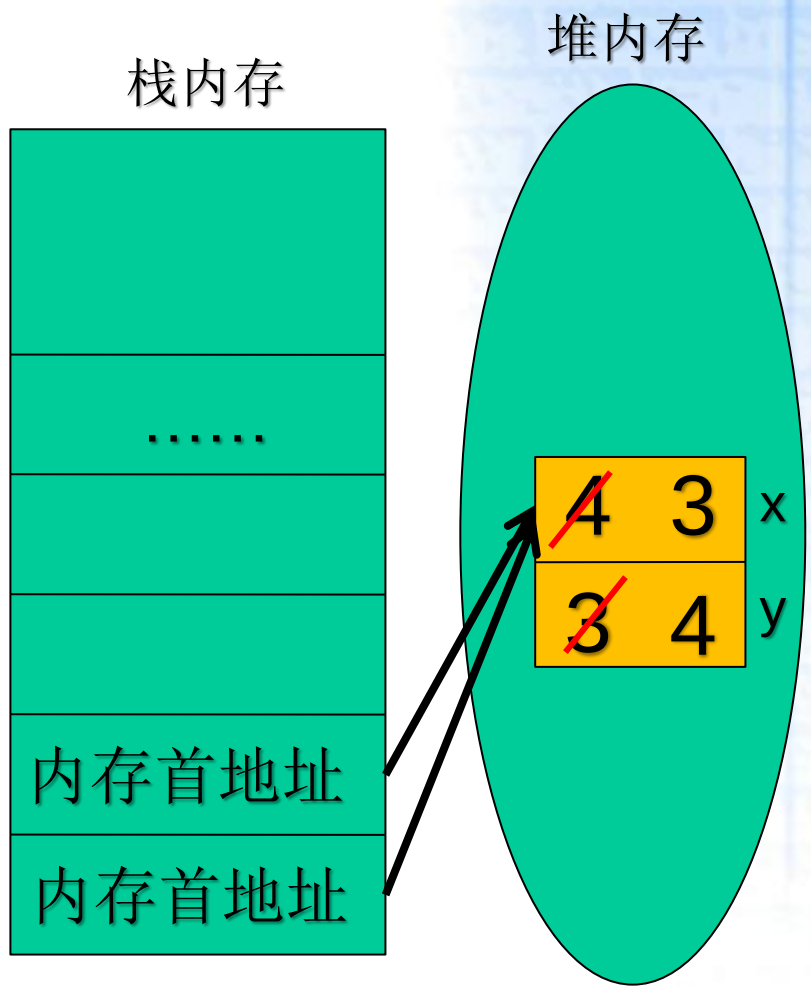
- 在Java中，传参时，都是以传值的方式进行：对于基本数据类型，传递的是数据的拷贝；对于引用类型，传递的是引用的拷贝。

```
public static void main(String[] args)
{
    int x = 3;
    int y = 4;
    change(x,y);
    System.out.println ("x="+x+", "+"y="+y+";");
}
public static void change(int x, int y)
{
    x = x+y;
    y = x-y;
    x = x-y;
}
```



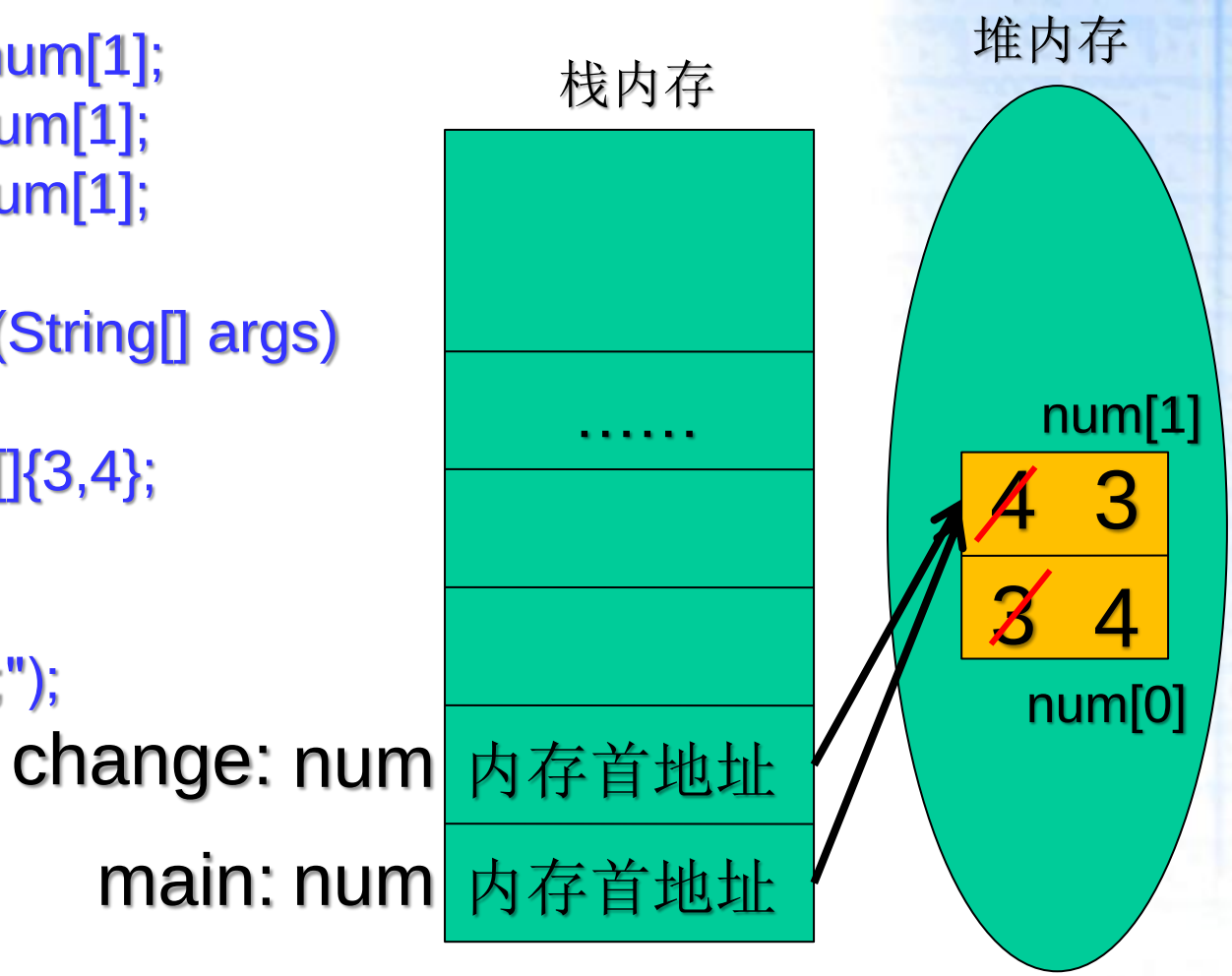
参数传值

```
public static void change(Point pt)
{
    pt.x = pt.x+pt.y;
    pt.y = pt.x-pt.y;
    pt.x = pt.x-pt.y;
}
public static void main(String[] args)
{
    Point pt = new Point();
    pt.x = 4;
    pt.y = 3;
    change(pt);
    打印pt.x，pt.y
}
```



参数传值

```
public static void change(int[] num)
{
    num[0] = num[0]+num[1];
    num[1] = num[0]-num[1];
    num[0] = num[0]-num[1];
}
public static void main(String[] args)
{
    int[] num = new int[]{3,4};
    change(num);
    System.out.println
(num[0]+", "+num[1]+";");
}
```



Test_01.java

可变参数

- public void fn(double a, **int … x**)
  - 参数依次为double类型的a，int类型的x[1]、x[2]…x[n]（其长度为x.length）
- 例：

  求平均数，可以定义一个函数

  double getAverage(int num, double …x)

  应用：

  getAverage(3, 5.0, 19.0, 22.0);

  getAverage(5, 11.0, 22.5, 29.3, 17.0, 33.5);

本章总结

- 本章学习了Java语言的一些实用系统类
  - 抽象类及其特点
  - 接口及其特点
  - 抽象类和接口的异同
  - “==”与equals的区别

Class is Dismissed!

